



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Semestre 1-2025
Técnicas Avanzadas de Programación



Asignación #2: Heaps

Análisis de Operaciones en heaps Binario, Binomial y Fibonacci

Estudiante:

Erimar Reis CI 29.743.464

Caracas, 27 de Mayo de 2025

Introducción

Un **Heap** es una estructura de datos similar a un árbol que mantiene un orden específico entre sus elementos sin ser un simple arreglo, permitiendo un acceso eficiente al valor mínimo o máximo del conjunto. Su diseño garantiza que las operaciones fundamentales como inserción, eliminación y consulta de elemento sean ejecutadas en el menor tiempo posible, lo que los convierte en una herramienta útil para la gestión de grandes cantidades de datos en algoritmos. Gracias a sus características particulares estas estructuras son ampliamente utilizadas para resolver problemas como planificación de tareas, algoritmos de rutas óptimas y administración de memoria. En este informe, implementamos y comparamos los tres tipos de heaps vistos en clase: Binario, Binomial y Fibonacci, analizando su comportamiento en las operaciones esenciales como inserción de elementos, consultas y unión de dos heaps.

Implementaciones

■ Heap Binario

Es una estructura implementada mediante un arreglo dinámico `vector<Nodo>`, lo que permite gestionar eficientemente las operaciones de inserción y extracción. Para preservar las características del heap se emplean dos operaciones clave: flotar, que consiste en ir "elevando" el nodo desde las hojas hasta la raíz hasta que este nuevo nodo permita que el heap cumpla sus características, y huir, que se emplea para extraer el mínimo.

Entre sus ventajas teóricas destaca su eficiencia para insertar y extraer elementos con complejidad $\mathcal{O}(\log n)$, lo cual garantiza tiempos de ejecución competitivos frente a implementaciones estáticas. Sin embargo, una de sus mayores limitaciones radica en la operación unión, ya que no existe un procedimiento eficiente para fusionar dos heaps sin reconstruirlos, la única opción viable (y la aplicada en esta implementación) es combinar los arreglos y aplicar `heapify`, lo cual implica una complejidad $\mathcal{O}(n + m)$.

■ Heap Binomial

Es una estructura que colecciona árboles binomiales organizados según su grado, cada árbol está compuesto por nodos interconectados mediante punteros hacia padres, hijos y hermanos, permitiendo realizar operaciones eficientes en comparación con el heap binario.

Una de sus principales características es la unión de heaps, que sí está optimizada para ser eficiente gracias a su mecanismo de fusión el cual combina estructuras similares en un tiempo $\mathcal{O}(\log n)$, esto lo convierte en una opción robusta cuando se requieren múltiples uniones. No obstante, la inserción en este heap, aunque funcional, introduce una complejidad $\mathcal{O}(\log n)$ debido a la consolidación del árbol luego de cada inserción, lo que en algunos casos impacta significativamente la eficiencia.

■ Heap Fibonacci

Este heap es una estructura más avanzada que se apoya en las listas circulares, cada nivel de cada subarbol está enlazado mediante punteros que forman listas circulares. Esta implementación se destaca por su capacidad de amortizar costos mediante la postergación de consolidaciones, haciendo que la inserción y la unión sean sus principales fortalezas, ambas con un costo $\mathcal{O}(1)$ amortizado.

Sin embargo, su complejidad estructural introduce ciertos inconvenientes. El sobrecoste de memoria es mayor en comparación con otros modelos, esto debido a que cada nodo contiene múltiples punteros. Además, en tamaños pequeños de datos la amortización puede no reflejar beneficios, por lo que para pocos elementos puede no ser la opción óptima.

Metodología Experimental

Para evaluar el rendimiento de los Heaps se abordaron los tres experimentos planteados en el enunciado, permitiendo analizar su comportamiento en distintas operaciones clave. La metodología sigue un enfoque cuantitativo,

midiendo los tiempos de ejecución bajo las mismas condiciones asegurando que los datos utilizados siempre sean los mismos en cada implementación para garantizar comparaciones precisas.

Cada heap está implementado en un archivo .cpp independiente. Los experimentos utilizan una semilla para generar los datos necesarios y el archivo principal (main) de cada implementación ejecuta las pruebas según los valores de N ingresados. Para garantizar consistencia, un script en Python compila y ejecuta los tres archivos .cpp con la misma semilla y valores de N , este script realiza tres ejecuciones por archivo, cada una con una semilla distinta, y almacena los datos generados en archivos .txt separados. Posteriormente, un segundo script procesa estos archivos, calcula los valores promediados y genera un nuevo .txt con los resultados consolidados para cada heap, además de graficar individualmente cada experimento. Finalmente, un tercer script analiza los resultados de los heaps con los tiempos promediados y genera tres gráficas comparativas para evaluar el rendimiento de cada heap.

Para los experimentos se usaron las semillas [123456789, 9876, 555] y los valores de N [100, 500, 1000, 5000, 10000, -1]

Análisis de resultados

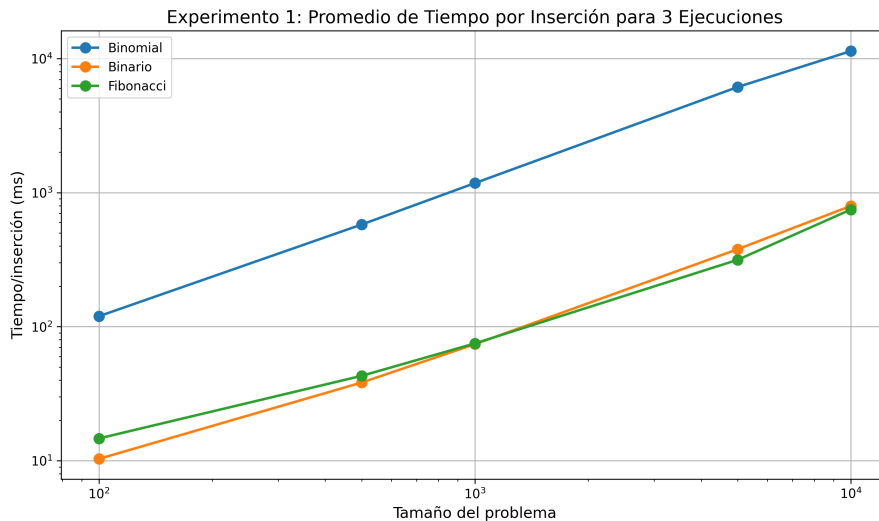
Para evaluar el rendimiento de los algoritmos implementados se realizaron pruebas controladas en las que se midieron los tiempos de ejecución en microsegundos. A continuación se resume los resultados obtenidos:

Experimento 1: Inserciones masivas

En este experimento se realizaron múltiples inserciones de números aleatorios en cada Heap y se midió el tiempo promedio por inserción. Se observó el siguiente comportamiento:

Cuadro 1: Comparación de tiempos en inserción (Experimento 1)

M	Binario		Binomial		Fibonacci	
	T. total (ms)	T./ins (ms)	T. total (ms)	T./ins (ms)	T. total (ms)	T./ins (ms)
100	10.33	0.1033	119.67	1.1967	14.67	0.1467
500	38.33	0.0767	578.33	1.1567	43.00	0.0860
1,000	74.00	0.0740	1177.33	1.1773	75.00	0.0750
5,000	378.33	0.0757	6133.33	1.2267	315.33	0.0631
10,000	795.33	0.0795	11365.33	1.1365	747.67	0.0748



De estos resultados observamos que:

- El Heap Binario y Heap Fibonacci presentan un escalamiento más eficiente con tiempos inferiores a 0.1 microsegundos por inserción.
- El Heap Binomial es mucho más lento en comparación con los otros heaps, debido a la consolidación de árboles tras cada inserción.

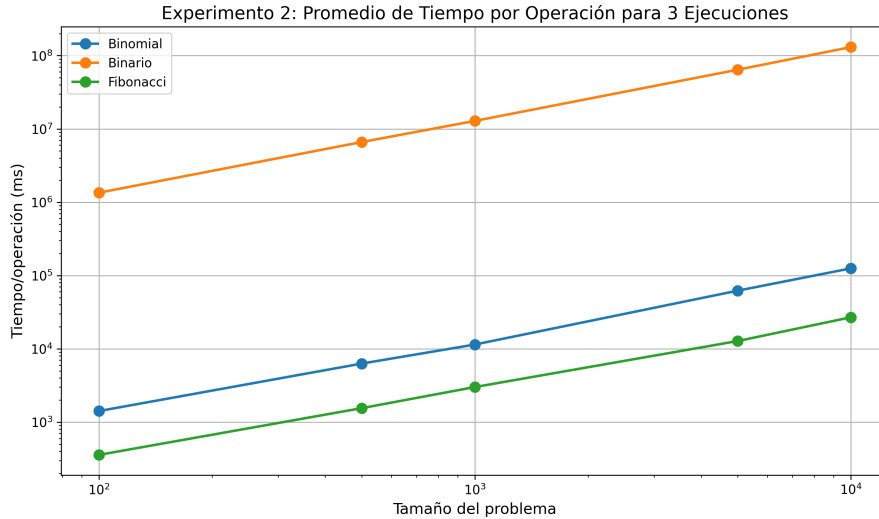
Este comportamiento confirma que las inserciones en Binomial son costosas, mientras que la amortización de Fibonacci permite tiempos de ejecución más estables a medida que el número de elementos aumenta.

Experimento 2: Operaciones aleatorias

Se aplicaron operaciones aleatorias en los tres Heaps, incluyendo inserción, consulta del mínimo, extracción y unión. Los tiempos obtenidos fueron los siguientes:

Cuadro 2: Comparación de tiempos en operaciones mixtas (Experimento 2)

O	Binario		Binomial		Fibonacci	
	T. total (ms)	T./op (ms)	T. total (ms)	T./op (ms)	T. total (ms)	T./op (ms)
100	1,349,217.33	13,492.20	1,417.00	14.17	357.67	3.58
500	6,619,727.67	13,239.43	6,292.67	12.59	1,551.00	3.10
1,000	12,866,573.67	12,866.57	11,446.67	11.45	3,011.33	3.01
5,000	64,180,665.00	12,836.13	62,138.67	12.43	12,771.00	2.55
10,000	130,665,532.67	13,066.57	125,187.33	12.52	26,920.67	2.69



De estos resultados observamos que:

- Heap Fibonacci resultó ser 4-5 veces más rápido que Heap Binomial en la ejecución de operaciones mixtas. Esto se debe a su amortización en inserciones y uniones.
- Heap Binario tuvo una degradación significativa en su rendimiento, especialmente en la operación de unión, que tiene una complejidad de $\mathcal{O}(n + m)$, Este factor lo hace ineficiente en pruebas con múltiples combinaciones.

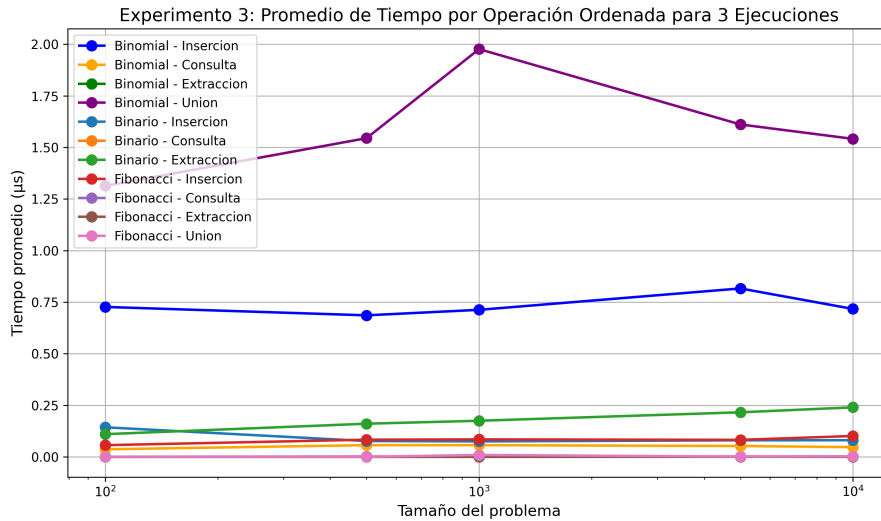
Los resultados muestran que la estructura de Fibonacci le otorga una ventaja en cargas de trabajo dinámicas, mientras que la implementación Binomial mantiene un tiempo estable pero con latencia en inserciones.

Experimento 3: Secuencia de Operaciones

Se evaluó una serie de operaciones secuenciales aplicadas a cada Heap, incluyendo inserción, consulta, extracción y unión. Los resultados fueron los siguientes:

Cuadro 3: Comparación de tiempos por operación (Experimento 3)

N	Estructura	Inserción (μ s)	Consulta (μ s)	Extracción (μ s)	Unión (μ s)
100	Binario	0.1433	0.0000	0.1100	0.4800
	Binomial	0.7267	0.0367	0.0000	1.3133
	Fibonacci	0.0567	0.0000	0.0000	0.0000
500	Binario	0.0767	0.0020	0.1607	1.3433
	Binomial	0.6860	0.0567	0.0000	1.5447
	Fibonacci	0.0833	0.0013	0.0000	0.0000
1,000	Binario	0.0750	0.0020	0.1750	2.4050
	Binomial	0.7127	0.0567	0.0000	1.9763
	Fibonacci	0.0847	0.0017	0.0003	0.0093
5,000	Binario	0.0802	0.0020	0.2160	12.0050
	Binomial	0.8163	0.0525	0.0005	1.6111
	Fibonacci	0.0825	0.0016	0.0004	0.0009
10,000	Binario	0.0807	0.0020	0.2400	25.1873
	Binomial	0.7170	0.0472	0.0004	1.5413
	Fibonacci	0.1012	0.0017	0.0004	0.0027



De estos resultados observamos que:

- En la operación de unión, Heap Fibonacci es claramente superior, con una complejidad de $\mathcal{O}(1)$, superando ampliamente a Binomial $\mathcal{O}(\log n)$ y Binario $\mathcal{O}(n)$.
- Para extracciones, Binomial y Fibonacci presentan los mejores tiempos $< 0,05$ microsegundos, lo que los hace adecuados para escenarios que requieren recuperación eficiente de datos.

Es claro que el rendimiento del Heap Binario se ve afectado por la falta de una unión eficiente, mientras que Binomial y Fibonacci son capaces de amortizar este costo.

Comparación del rendimiento

Al comparar las implementaciones, se destacan los siguientes puntos clave:

- El Heap Binario es eficiente en inserciones individuales, pero presenta serias limitaciones en operaciones de unión y escenarios dinámicos.
- El Heap Binomial mantiene una estructura estable, pero la consolidación tras cada inserción introduce latencia adicional considerable.
- El Heap Fibonacci es la opción más eficiente en operaciones mixtas y especialmente en uniones, gracias a su amortización en inserciones y aplazo de consolidaciones.

Complejidad Práctica vs. Teórica

Los resultados experimentales reflejan diferencias significativas entre la complejidad teórica y el desempeño real de cada tipo de Heap, especialmente en escenarios con cargas de trabajo variadas.

- Heap Binario, aunque teóricamente garantiza $\mathcal{O}(\log n)$ en inserciones y extracciones, su rendimiento se ve afectado en operaciones de unión, donde el costo de reconstrucción completa se asemeja más a $\mathcal{O}(n \log n)$ introduce una penalización considerable en pruebas grandes.
- Heap Binomial mantiene la complejidad teórica de $\mathcal{O}(\log n)$ en inserciones, pero en la práctica su constante de tiempo es alta debido a la consolidación inmediata de estructuras. Esto lo hace menos eficiente en inserciones frecuentes.
- Heap Fibonacci cumple con su complejidad amortizada $\mathcal{O}(1)$ en inserciones y uniones, lo que lo hace altamente eficiente. Sin embargo su consolidación aplazada puede introducir una latencia inesperada en ciertas situaciones.

Conclusiones

De este estudio se concluye que el Heap Binario es eficiente para grandes volúmenes de inserciones y extracciones, pero su desempeño se degrada en entornos con frecuentes uniones debido al alto costo de reconstrucción. El Heap Binomial se presenta como una opción equilibrada para operaciones mixtas, ofreciendo estabilidad en múltiples uniones, aunque la consolidación tras cada inserción introduce una carga significativa para la memoria en cargas masivas. Finalmente, el Heap Fibonacci destaca como la mejor alternativa para aplicaciones con un gran número de operaciones dinámicas, ya que optimiza las uniones y las inserciones masivas, siendo ideal para volúmenes de datos superiores a 10,000 elementos.

De este informe se destaca la relevancia de analizar estructuras de datos desde una perspectiva práctica y teórica para comprender sus fortalezas y limitaciones en distintos contextos.

Referencias

- [Binomial Heaps - Algorithm Tutor](#)
- [Binomial Heap in C++ - GeeksforGeeks](#)
- [Fibonacci Heap - Woodfrog](#)
- [Fibonacci Heap: Insertion and Union - GeeksforGeeks](#)
- [DeepSeek - AI Chat](#)